APPENDIX 1

RSpec and the Wider Ruby Ecosystem

RSpec does not exist in a vacuum. It's part of the wider Ruby ecosystem, and is designed to work well with existing Ruby tools. In this appendix, you'll see how to use RSpec effectively with two of Ruby's most important tools: Bundler and Rake. We'll also show you how to use parts of RSpec with other test frameworks.

Bundler

When you're using Bundler to manage your dependencies, there are a few different ways you can ensure that the correct versions of all your libraries get loaded:

- Call `Bundler.require` from your Ruby code
- Wrap every Ruby program with `bundle exec` at the command line
- Use Bundler's stand-alone mode

All of these techniques work with RSpec. Let's take a look at each one in turn.

The first option, `Bundler.require`, is convenient: it loads all your project's gems, so you don't have to remember to `require` each gem individually before use. But it has implications for your application's boot time and maintainability, as Myron points out in his blog post.[1]

The second option, `bundle exec`, is faster and avoids some of these maintainability pitfalls, but is still inefficient. Every time you run (for example) `bundle exec rspec`, Bundler spends time validating that you have all the right gem versions installed—even though this validation is only needed on the infrequent occasions that your project's gems change.

1. http://myronmars.to/n/dev-blog/2012/12/5-reasons-to-avoid-bundler-require